
METODOLOGIE LEAN E KANBAN

Michele Marchesi

marchesi@unica.it

**Corso di Ingegneria del Software
A.A. 2025/26**



Lean Development

- ◆ Negli ultimi 40 anni il "Lean Development":
 - ha avuto un enorme impatto economico
 - ha trasformato sostanzialmente molte industrie
- ◆ La Toyota è stata leader nell'attuazione delle pratiche Lean negli anni '80
- ◆ Lo sviluppo software Lean (LSD) applica l'approccio Lean alla produzione del software



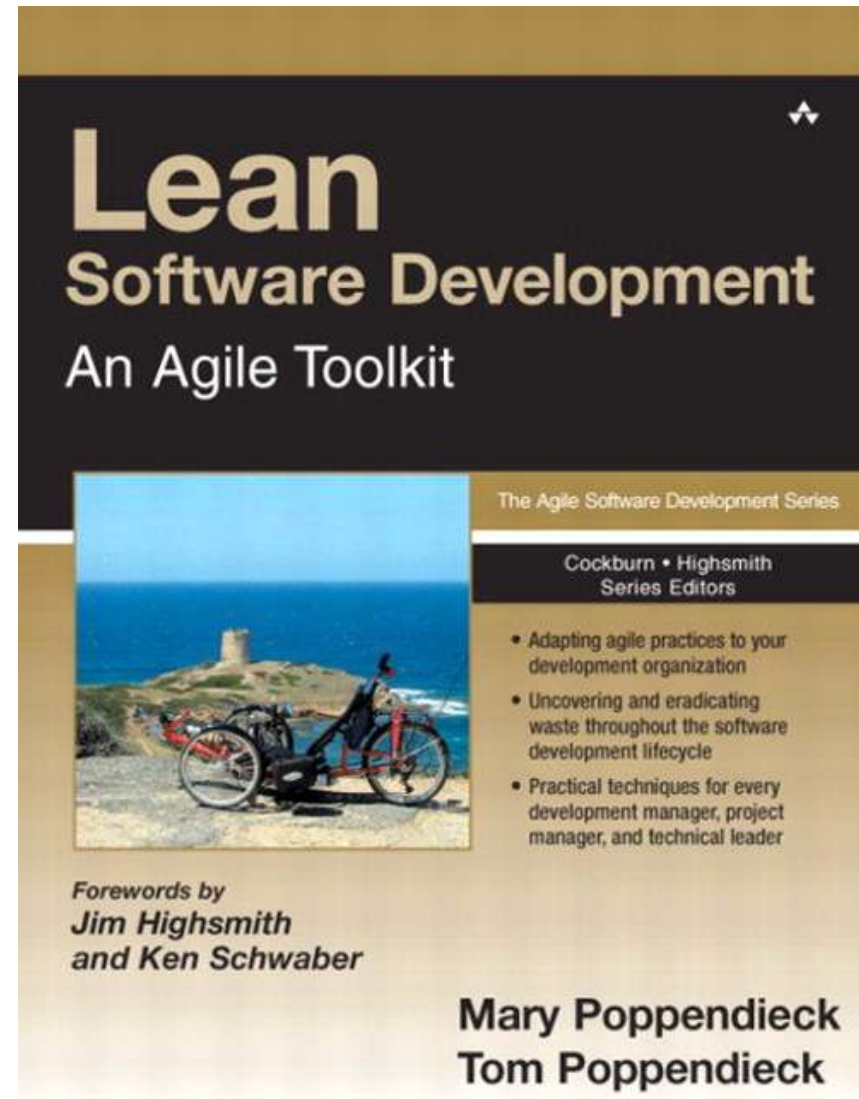
Lean Software Development (LSD)

- ◆ L'approccio LSD non si concentra sulle pratiche di sviluppo software, ma piuttosto su *come e quando eseguire il lavoro*
- ◆ L'applicazione dei principi Lean è diversa a seconda dei contesti industriali, così come è diversa a seconda dei contesti di sviluppo software
- ◆ Non tutte le pratiche del *Lean Manufacturing* possono essere applicate al software
- ◆ L'approccio LSD prevede 7 principi e 22 strumenti



Lean Software Development ***(Poppendieck, 2003)***

- **Applicazione dell'approccio Lean allo sviluppo del software**
- **Metodologia proposta da Mary e Tom Poppendieck**
- **Presentata la prima volta a XP2002 (Alghero)**



Panel a XP2002

Da sx a dx:

- Ken Schwaber
- Mary Poppendieck
- Martin Fowler
- Kent Beck
- David Parnas



I 7 principi Lean (Poppendieck, 2003)

- 1. Eliminate Waste** – *Eliminare gli sprechi*
- 2. Amplify Learning** – *Ampliare l'apprendimento*
- 3. Decide as late as possible** – *Decidere il più tardi possibile*
- 4. Deliver as fast as possible** – *Consegnare il più velocemente possibile*
- 5. Empower the team** – *Date potere al team*
- 6. Build integrity in** – *Creare sistemi intrinsecamente coesi*
- 7. See the whole** – *Considerare il tutto*



1. *Eliminate Waste*

- ◆ Che cosa sono gli sprechi (waste)?:
 - Tutto ciò che non aggiunge valore al prodotto
 - In pratica, ***il cliente sarebbe ugualmente soddisfatto senza di essi***
- ◆ Prima direttiva del **Lean Thinking**:
 - Creare *valore* per il cliente
 - Accrescere il flusso di valore (*Value Stream*)



Waste (Muda)

◆ **Nell'industria**

- Magazzino
- Sovra-lavorazioni
- Sovra-produzione
- Trasporto (esterno)
- Attese
- Moto (interno)
- Difetti

◆ **Nel software**

- Lavoro incompleto
- Produzione di carta
- Funzionalità extra non necessarie
- Passaggi di mano
- Attesa dell'informazione necessaria
- Cambio compiti & moto
- Difetti



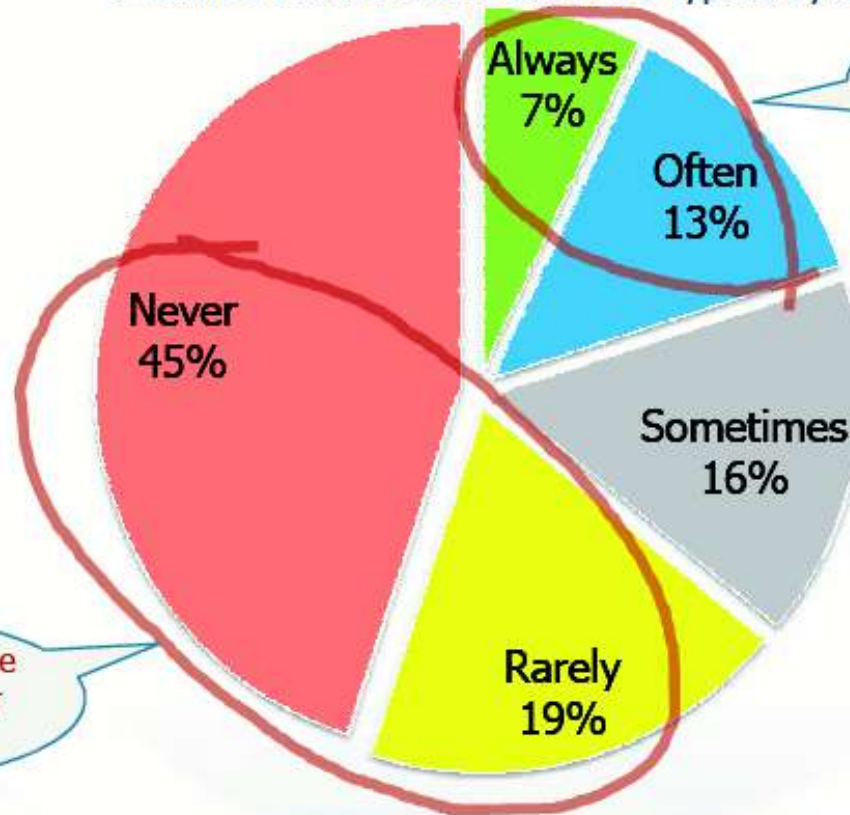
Sprechi dello sviluppo software

- ◆ **Lavoro svolto parzialmente** e non ancora produttivo
- ◆ **Processi inutili** (ad es. produzione di documentazione inutile, necessità di autorizzazioni inutili, riunioni inutili, stime inutili)
- ◆ **Funzionalità inutili** (occorre seguire il principio del “*fa la cosa più semplice che possa funzionare*”) ← **ricorda qualcosa?**
- ◆ **Cambio di compiti** (persone che lavorano contemporaneamente a più progetti e cambiano spesso da uno all'altro)
- ◆ **Attese**, dovute a ritardi esterni e/o a ritardi interni
- ◆ **Moto**, spostamenti e tempo per rispondere a domande, trovare informazioni necessarie, passaggio di documenti e manufatti (anche elettronici)
- ◆ **Difetti ed errori** da correggere



La più grande sorgente di sprechi

Features and functions used in a typical system:



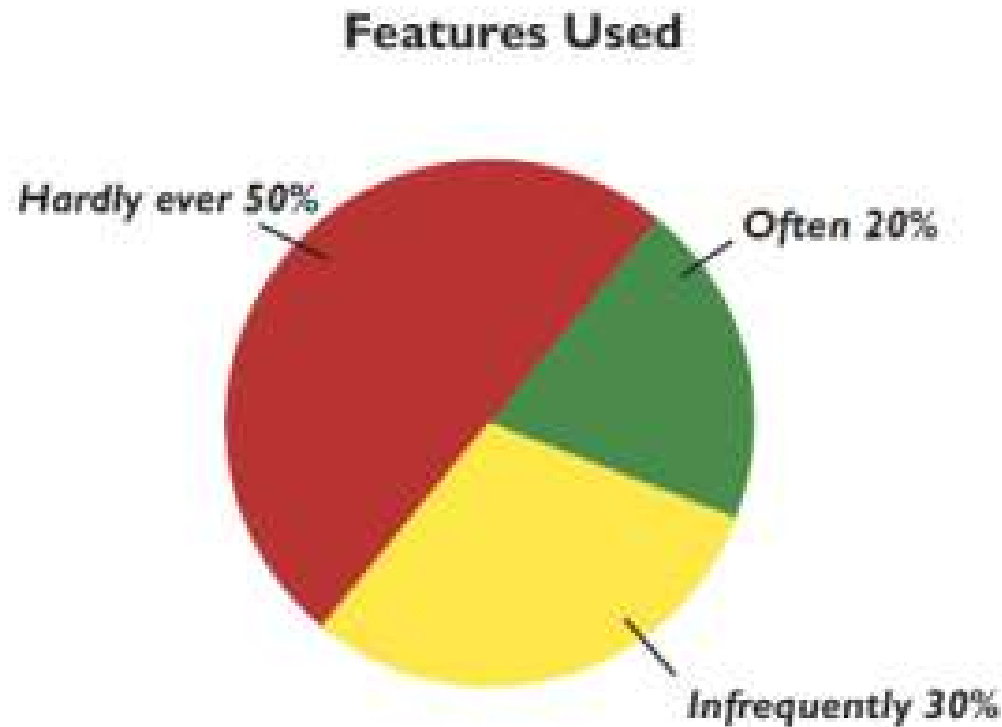
Only 1/5 of the stuff we build is used often or always!

2/3 of the stuff we build is rarely or never used!



Source: Standish Group Study Reported at XP2002
by Jim Johnson, Chairman

La più grande sorgente di sprechi (aggiornato al 2017)



Features commonly seen in custom applications: How often are they actually used? Remember, the more objectives are attached to a project, the more people become involved, and the more complex it becomes.

2. *Amplify learning*

- ◆ Lo sviluppo software è un'attività di progetto, e non di produzione
 - E' quindi molto variabile e incerta all'inizio, perché occorre verificare le alternative, e perché il grado di rischio è alto
- ◆ Occorre quindi utilizzare ciò che si apprende dal cliente, dal team stesso e dal software sviluppato, massimizzandone il feedback e sviluppando per piccoli lotti (cioè per funzionalità elementari), che migliorano:
 - la qualità
 - la comunicazione tra sviluppatori
 - l'utilizzo delle risorse



3. *Decide as late as possible*

- ◆ Nel software, il costo di correzione di errori nei requisiti cresce esponenzialmente a seconda che la correzione sia fatta nelle fasi di analisi, progetto, codifica, o dopo il rilascio.
- ◆ Lo sviluppo iterativo e incrementale consente di ritardare molte decisioni di progetto
- ◆ Infatti, tutti gli *incrementi* (features, User Stories) non ancora implementati si possono modificare/cancellare a costo zero
- ◆ Ciò permette di risparmiare risorse in caso di modifiche a vincoli/requisiti che avvengono dopo l'inizio dello sviluppo del sistema.



3. Decide as late as possible: opzioni

- ◆ Ogni scelta di implementazione va considerata come se fosse un'opzione, con un costo e dei benefici associati, da calcolare e di cui tenere conto:
 - Le stesse funzionalità da implementare possono essere associate a opzioni.
 - Le varie possibilità di implementazione possono essere anch'esse associate a più opzioni alternative.
 - La scelta di quale opzione esercitare effettivamente va fatta il più tardi possibile, quando i costi e i benefici delle alternative sono più stabili e chiari.



4. Deliver as fast as possible

- ◆ Dall'idea al prodotto finito deve passare il minor tempo possibile...
- ◆ ...ma senza avere troppa fretta, compromettendo la qualità!
- ◆ Gli sviluppatori devono avere sempre qualcosa da fare, e devono saperlo da soli, senza che qualcuno glielo dica.
- ◆ Gli strumenti per minimizzare i tempi di lavorazione sono due: approccio “pull” e teoria delle code



Minimizzare i tempi di lavorazione

- ♦ **Approccio “pull”**: chi lavora a una fase della lavorazione inizia solo se la fase precedente termina e rende disponibile un altro prodotto:
 - lo stato della lavorazione viene visualizzato in un tabellone Kanban
 - gli addetti alle lavorazioni sono bilanciati in modo da non avere persone troppo occupate e persone libere
- ♦ **Uso della teoria delle code e degli ostacoli**, che considera i tempi di transito e di attesa di elementi entro delle code.
- ♦ In pratica, il tempo di consegna (tempo impiegato da un elemento per percorrere tutta la coda) va ridotto al minimo, riducendo arrivi di blocchi troppo grandi di funzionalità e trovando ed eliminando i colli di bottiglia che rallentano lo sviluppo



5. *Empower the team*

- ◆ L'approccio Lean supera il Fordismo, cioè il cosiddetto “approccio scientifico” alla pianificazione industriale in cui le persone sono guidate in modo dettagliato dai loro capi
- ◆ L'approccio Lean, invece, si basa sulle assunzioni alternative:
 - Un'organizzazione matura considera il sistema come un tutto, e non si focalizza a ottimizzare parti disaggregate
 - Un'organizzazione matura si focalizza su un apprendimento efficace, e dà alle persone che fanno il lavoro il potere di prendere decisioni



6. *Build integrity in*

- ◆ Un sistema di successo deve avere integrità (coerenza) percepita e integrità concettuale, e avere quindi l'integrità “costruita dentro”. I tre requisiti fondamentali per ottenere ciò sono:
 - Elevata competenza del dominio applicativo in tutto il team
 - Accettazione del cambiamento come norma, e capacità di adattarsi a cambiamenti di progetto emergenti in corso d'opera
 - Un ambiente che favorisca la comunicazione, integrando persone, strumenti e informazione
- ◆ Tra i principali strumenti per ottenere l'integrità ci sono il testing continuo e automatizzato, e il refactoring



7. *See the whole*

- ◆ Occorre migliorare e ottimizzare il risultato complessivo, non le singole parti.
 - Ciò si può ottenere promuovendo soluzioni che portino ai risultati voluti, eliminandone gli effetti secondari che possono alla lunga indebolire tali risultati.
- ◆ Occorre essere attenti a curare le cause dei problemi, e non semplicemente i sintomi.
 - Curare i sintomi è di solito più facile, ma non porta benefici durevoli.
- ◆ Si ha spesso la tentazione di dividere un sistema complesso in parti più semplici, e gestirle come se fossero indipendenti.
 - Migliorando le singole parti, però, non è detto che si migliori il tutto



Agile vs. Lean

- ◆ La metodologia Lean è più completa rispetto ad una Agile, perché può essere applicata in molti campi
- ◆ La metodologia Agile, al contrario, è nata per il software, anche se recentemente tende a essere applicata anche ad altri settori
- ◆ Nel software, la metodologia Lean è spesso vista come un tipo di metodologia agile
- ◆ I due approcci (o mentalità) condividono molte idee, ma sono diversi



Agile & Lean: obiettivi diversi

- ◆ L'obiettivo principale del Lean è di **aumentare il valore dato al cliente**
 - Questo è ottenuto principalmente dalla **riduzione degli sprechi**
- ◆ L'obiettivo principale delle Metodologie Agili è quello di **essere in grado di gestire i continui cambiamenti nei requisiti**
 - Questo è ottenuto attraverso: **sviluppo iterativo e incrementale, semplicità, TDD, refactoring e altri strumenti**



Agile & Lean – storia

- ◆ Molti fautori delle AM sono stati influenzati dal Lean manufacturing:
 - Concentrarsi sul valore di business per il cliente
 - Riduzione del tempo di ciclo (→ iterazioni brevi)
 - Teoria delle opzioni (processo decisionale utilizzato in matematica finanziaria)
 - Responsabilizzazione del team
- ◆ Il LSD ha adottato diversi strumenti dalle pratiche agili:
 - Testing, Refactoring
 - Iterazioni brevi, feedback, semplicità



Agile vs. Lean

- ◆ **Principi Lean:**

1. Add nothing but value (Eliminate Waste)
2. Center on the people who add value
3. Flow value from demand
4. Optimize across organizations

- ◆ Le AM affrontano i punti 2 e 3 e parte del punto 4, ma non il punto 1
- ◆ Le AM prevedono un processo specifico, e spesso aggiungono ad esso delle pratiche di sviluppo (XP, FDD)
- ◆ Essere Agile non significa necessariamente essere Lean e viceversa (il Waterfall può essere implementato anche in modo Lean)



Agile & Lean – caratteristiche comuni

- ◆ Approcci incentrati sulle persone
 - ◆ Responsabilizzazione del team
 - ◆ Pianificazione adattiva
 - ◆ Miglioramento continuo
-
- ◆ In pratica, le persone che adottano le metodologie Agili tendono ad allinearsi anche ai principi Lean
 - ◆ Viceversa, le persone che adottano l'approccio LSD seguono in genere almeno alcune pratiche agili



Kanban

- ◆ Kanban significa “*segnale visuale*” in giapponese
- ◆ È uno strumento del Lean Manufacturing
- ◆ Destinato a facilitare la condivisione di informazioni...
 - ...ma anche a controllare il flusso del lavoro
- ◆ Un numero limitato di tag Kanban può limitare il numero di elementi che possono circolare
- ◆ Nella figura, i contenitori di pezzi sono etichettati, e la barra colorata dietro evidenzia subito se vanno riassortiti!



Kanban

- ◆ Un numero limitato di tag Kanban può limitare il numero di elementi che possono circolare
 - Esempio: controllo del numero di visitatori presso i giardini del Palazzo imperiale a Tokio:
 - Le tessere sono gratis, ma il loro numero è limitato
 - All'uscita, sono restituite e riusate da altri visitatori



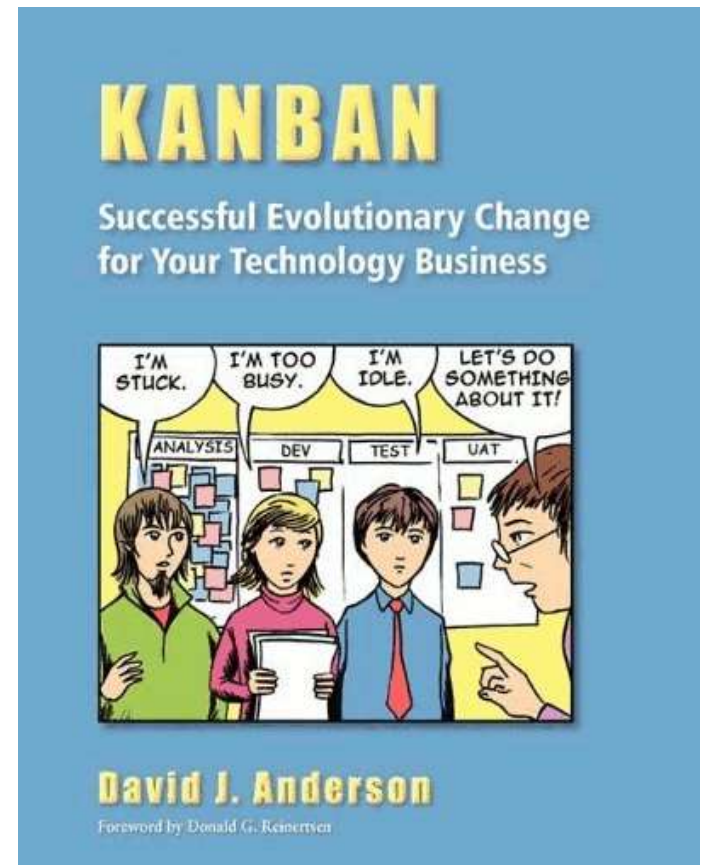
Kanban nello sviluppo software

- ◆ Il Kanban è stato inizialmente introdotto nel Lean Software Development attraverso l'uso di un tabellone per poter visualizzare lo stato di sviluppo
- ◆ Fu successivamente usato per controllare il flusso del lavoro
- ◆ Ora, il Kanban è in continua evoluzione come "metodo Kanban", per implementare effettivamente l'approccio Lean
 - “Kanban”, “Kanban board”, “Kanban Method”, “Scrumban”



Libro di Anderson

- ◆ Il miglior libro di riferimento sul metodo Kanban
- ◆ Pubblicato nel 2010
- ◆ Uno dei Best-Sellers Amazon nell'ambito dello sviluppo di software
- ◆ Motivo di interesse per il Kanban: il suo rapido successo



Kanban – successi

- ◆ Il metodo Kanban fu sperimentato per la prima volta nel 2004 allo XIT Sustained Engineering di Microsoft (uno degli 8 dipartimenti IT di Microsoft):
 - Un piccolo team in India, certificato CMM lev. 5
 - Supporta 80 applicazioni interne
 - Tempo di risposta medio: 155 giorni prima di introdurre il Kanban
 - Tempo di risposta medio: 14 giorni dopo il Kanban
- ◆ HP Printer division – 2006
 - Miglioramento di produttività 3,4 volte
 - 2 volte il miglioramento della qualità
 - Tempo di risposta da 9 mesi a 2 mesi



Kanban – storia

Periodo	Evento
1940-50	Toyota sviluppa il sistema Kanban per la produzione "just-in-time"
2003-04	David Anderson applica i concetti Lean/Kanban allo sviluppo software
2007	Primo utilizzo documentato di Kanban board in un team IT
2010	Pubblicazione del libro di Anderson
Oggi	Standard de-facto per gestione flusso, usato in combinazione con Scrum, DevOps, SAFe



Kanban – simulazione

- ◆ Articolo presentato a XP2012 (Malmoe, Svezia)
- ◆ Pubblicato su Lecture Notes in Business Information Processing, vol. 111, Editor C. Wohlin.

A comparative study of Scrum and Kanban approaches on a real case study using simulation

David J. Anderson¹, Giulio Concas², Maria Ilaria Lunesu², Michele Marchesi²,
and Hongyu Zhang³

¹ David J. Anderson&Associates inc., Seattle, WA, USA

² DIEE - Department of Electrical and Electronic Engineering,
University of Cagliari, Piazza D'Armi, 09123 Cagliari, Italy

³ School of Software, Tsinghua University, Beijing, China



Kanban – vantaggi

- ◆ Può essere utilizzato per lo sviluppo software, manutenzione o entrambi
- ◆ Può essere utilizzato da un gruppo di lavoro su più di un progetto nello stesso arco di tempo
- ◆ Può essere utilizzato con qualsiasi tipo di approccio e pratiche – nella prima applicazione, al MS XIT, il team ha usato CMM TSP/PSP
- ◆ Molto semplice da applicare, ma molto potente



Condizioni per l'applicazione dell'approccio Kanban

- ◆ Assiomi di Corey Ladas:

1. *È possibile dividere il lavoro in piccoli incrementi di valore aggiunto (small value-adding increments) che possono essere pianificati autonomamente.*
 - Questi incrementi sono le Features (caratteristiche), User Stories, Stories, Work Items, o Minimum Marketable Features (MMF) che abbiamo già visto! Il progetto termina quando tutte le sue features sono completate.
2. *È possibile sviluppare qualsiasi piccolo incremento di valore aggiunto in un flusso continuo, dai requisiti fino al rilascio.*
 - Così, il lavoro su ogni funzionalità, dalla sua specifica al rilascio, si realizza attraverso una serie di attività, svolte nello stesso ordine sequenziale.



I 6 Principi Fondamentali di Kanban

- 1) Visualizza il lavoro** – Rendi visibile ciò che è invisibile
- 2) Limita il Work In Progress (WIP)** – Concentrati su pochi compiti alla volta
- 3) Gestisci il flusso** – Ottimizza il movimento del lavoro
- 4) Rendi esplicite le politiche** – Regole chiare e condivise
- 5) Implementa cicli di feedback** – Segui ritmi di revisione e miglioramento
- 6) Migliora collaborativamente** – Evoluzione basata su dati ed esperimenti



Il tabellone Kanban (*Kanban Board*)

- ◆ Le colonne del tabellone rappresentano le attività del flusso di lavoro (decise dal team)
- ◆ Le funzionalità (*Features/US o ticket*) sono rappresentate su cartellini (*cards*)
- ◆ Una feature viene inserita nella colonna corrispondente all'attività dove lavoro su di essa è in corso (*work in progress*)
- ◆ Esiste un limite superiore del numero di feature che possono essere assegnate ad un'attività
- ◆ Quando c'è spazio e ci sono feature "finite" (*done*) nell'attività precedente, una feature può essere "tirata" (*pulled*)



Feature Card

Feature / story

Date when
added to board

Hard deadline
(if applicable)

2009-08-20 2009-09-30

(description)

★ = priority

★★ = panic



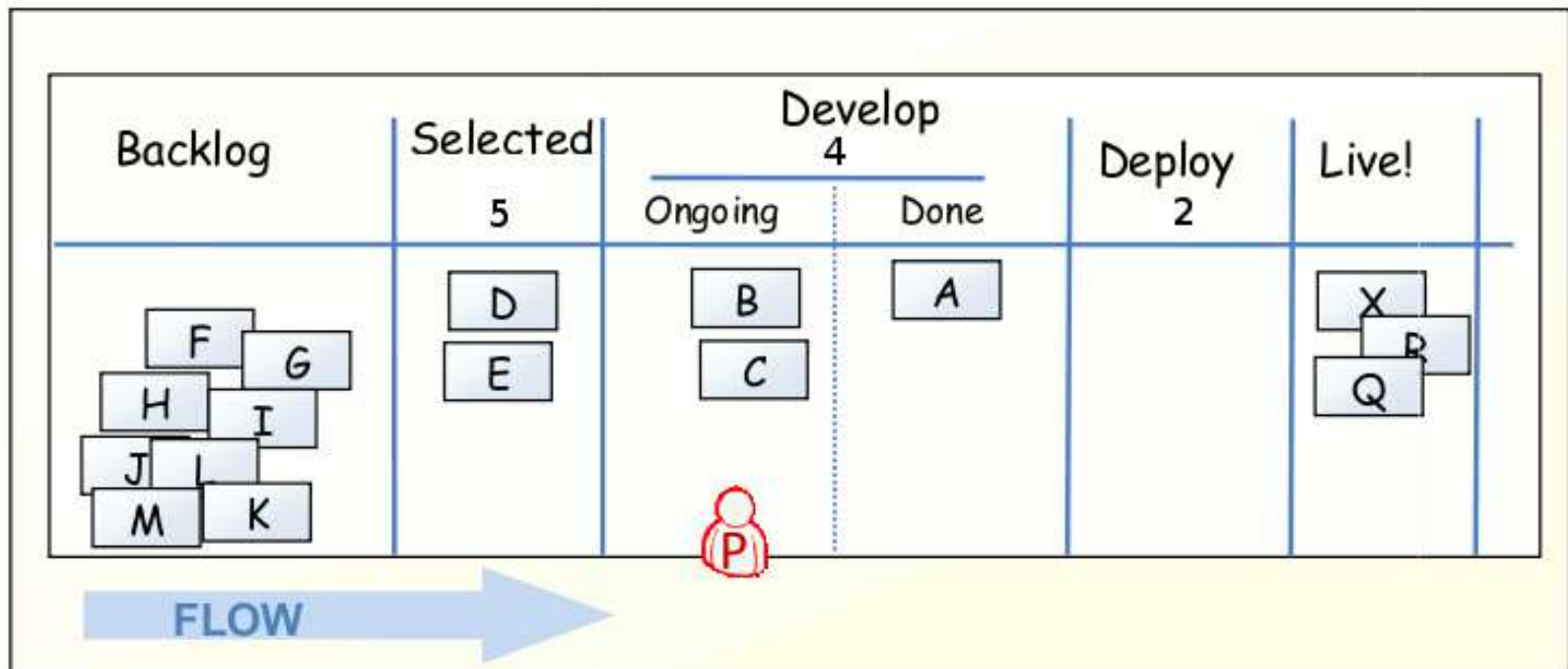
Who is analyzing /
testing right now

Elementi di una Card Efficace

- ◆ **Titolo chiaro:** "Aggiungere login con Google" (non "Login").
- ◆ **Descrizione sintetica:** Contesto e obiettivo.
- ◆ **Criteri di Accettazione:** Lista verificabile di condizioni per il "Done".
- ◆ **Responsabile:** Chi sta lavorando (max 1-2 per card in WIP-limited column).
- ◆ **Tag/Etichette:** Tipo (Feature/Bug), priorità, Feature/US di riferimento.
- ◆ **Data di scadenza** (opzionale): Per task in cui il tempo di rilascio è importante.



Un tabellone Kanban



Un tabellone Kanban: colonne

- ♦ **Backlog:** qui sono accumulate le feature da implementare
- ♦ **Selected:** qui sono poste le feature selezionate per iniziarna l'implementazione (max. 5 in questo esempio). Quando una di esse è spostata verso destra perché se ne inizia lo sviluppo, si può “tirarne” un'altra (pull) dall'insieme del Backlog. Questa è tipicamente quella di maggior priorità. Nell'esempio, si potrebbero “tirare” 3 feature entro questa colonna



Un tabellone Kanban: colonne

- ◆ **Develop:** le feature in corso di sviluppo, al massimo 4. Sono a loro volta divise in due sotto-colonne:
 - **Ongoing:** feature in corso di sviluppo;
 - **Done:** feature pronte per il rilascio.

Il vincolo sulle 4 feature al massimo si applica cumulativamente sulle feature Ongoing e Done. Nell'esempio, si può “tirare” subito un'altra feature dalla colonna “Selected”

- ◆ **Deploy:** le feature in corso di “deploy” (rilascio), al massimo 2. Poiché la colonna è vuota, si può “tirare” subito la feature “A” dalla sotto-colonna “Done” della colonna “Develop”.
- ◆ **Live!:** le feature rilasciate e funzionanti.



WIP - Work in Progress (o in Process)

- ♦ WIP: Numero di elementi di lavoro iniziati ma non ancora completati.
- ♦ Esempio: Se ho 5 feature "in sviluppo" contemporaneamente, il mio WIP = 5.
- ♦ Opinione errata: "Più cose faccio insieme, più veloce finisco"
- ♦ In realtà: Il multitasking aumenta i tempi di completamento e gli errori.
- ♦ Un elevato WIP è considerato uno spreco (waste) e ritarda la consegna del *valore* per il cliente



Perché limitare il WIP

- ◆ Scopo del tabellone kanban è limitare il Work in Progress (WIP, cioè il numero di feature contemporaneamente in lavorazione nella stessa attività)
- ◆ Il tempo necessario a completare una feature da quando è inserita nel backlog è chiamato “**Lead Time**”
- ◆ Il tempo necessario a completare una feature da quando ne inizia la lavorazione è chiamato – “**Cycle Time**”
- ◆ Il tabellone impone un limite sul numero di feature in corso in ogni attività, e quindi sul WIP
- ◆ Utilizzando il tabellone Kanban, i blocchi e i problemi sono presto e facilmente individuati
- ◆ L'altro strumento di controllo del WIP è il Cumulative Flow Diagram (descritto in seguito)



Cycle time e throughput (Produttività)

- ◆ **Throughput:** numero di elementi completati in un intervallo di tempo (es. in una settimana).
- ◆ Non è una metrica di prestazioni individuali! Serve a capire la capacità del sistema.
- ◆ Utilizzo: pianificare il backlog e comunicare previsioni agli stakeholder.
- ◆ Legge di Little: ***Cycle time = WIP / Throughput***
- ◆ A parità di throughput, ridurre il WIP riduce il Cycle Time.
- ◆ Risultato: Consegne più veloci e prevedibili.

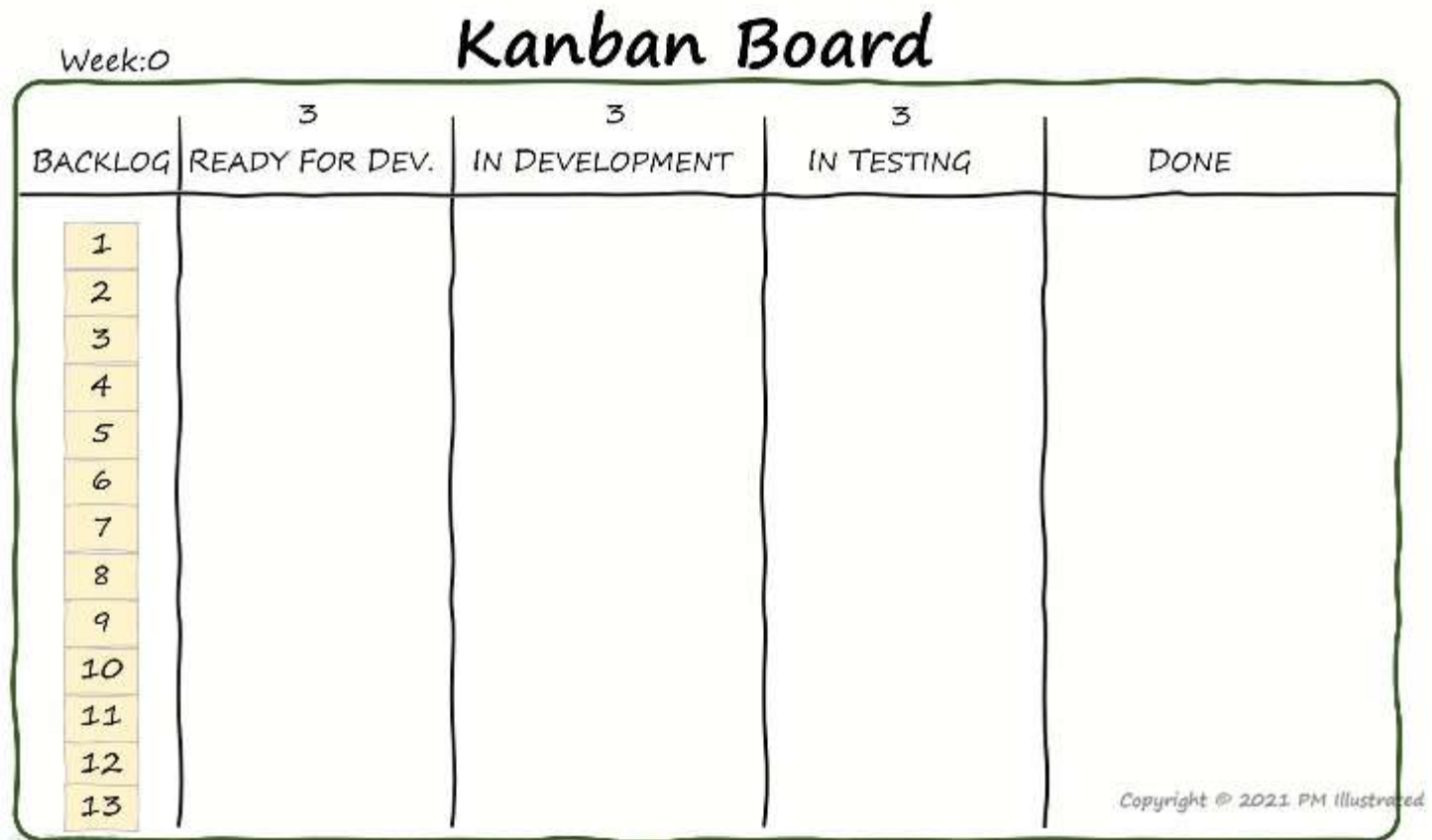


Perché limitare il WIP?

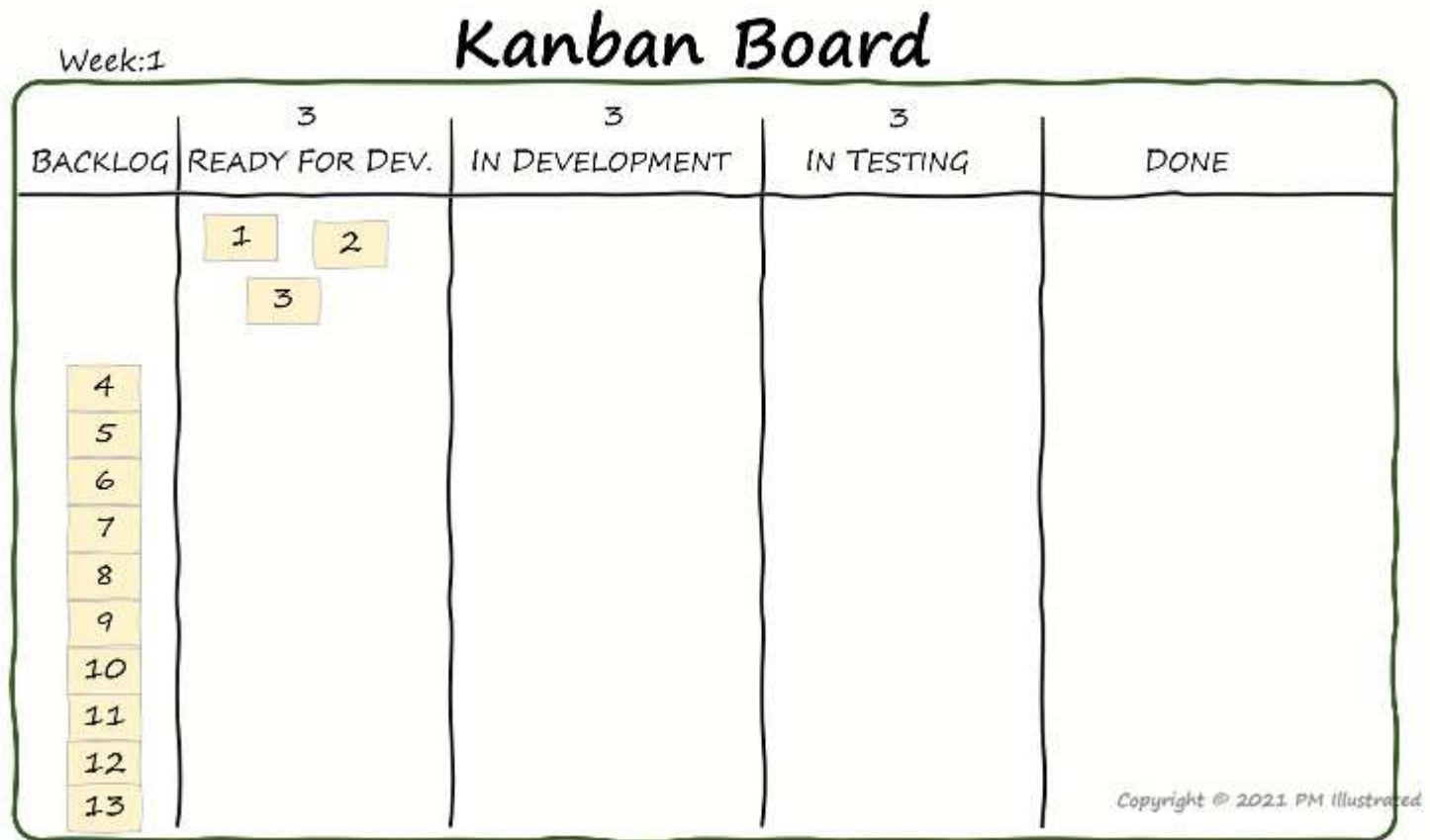
- ◆ Si riduce il *Context Switching*: il cervello umano impiega ~23 minuti per riprendere il focus dopo un'interruzione.
- ◆ Si migliora la qualità: meno fretta, più attenzione ai dettagli, quindi meno difetti.
- ◆ Si accelera il Feedback: task completati prima = feedback dagli utenti prima.
- ◆ Si identificano i colli di bottiglia: se una colonna è sempre piena, lì c'è un problema di processo.
- ◆ Si riduce lo stress del team: più focus su una cosa per volta.



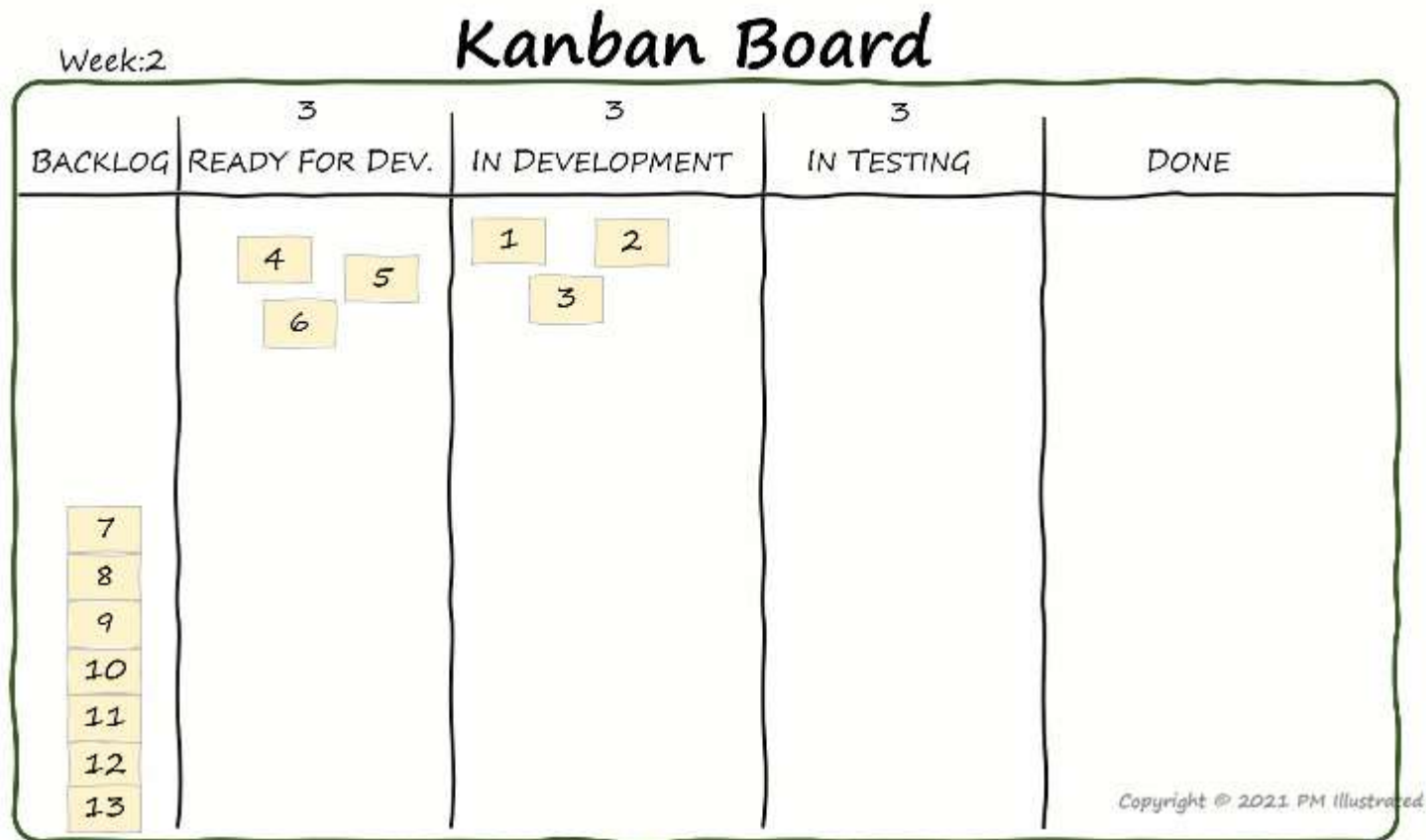
Esempio di funzionamento - 1



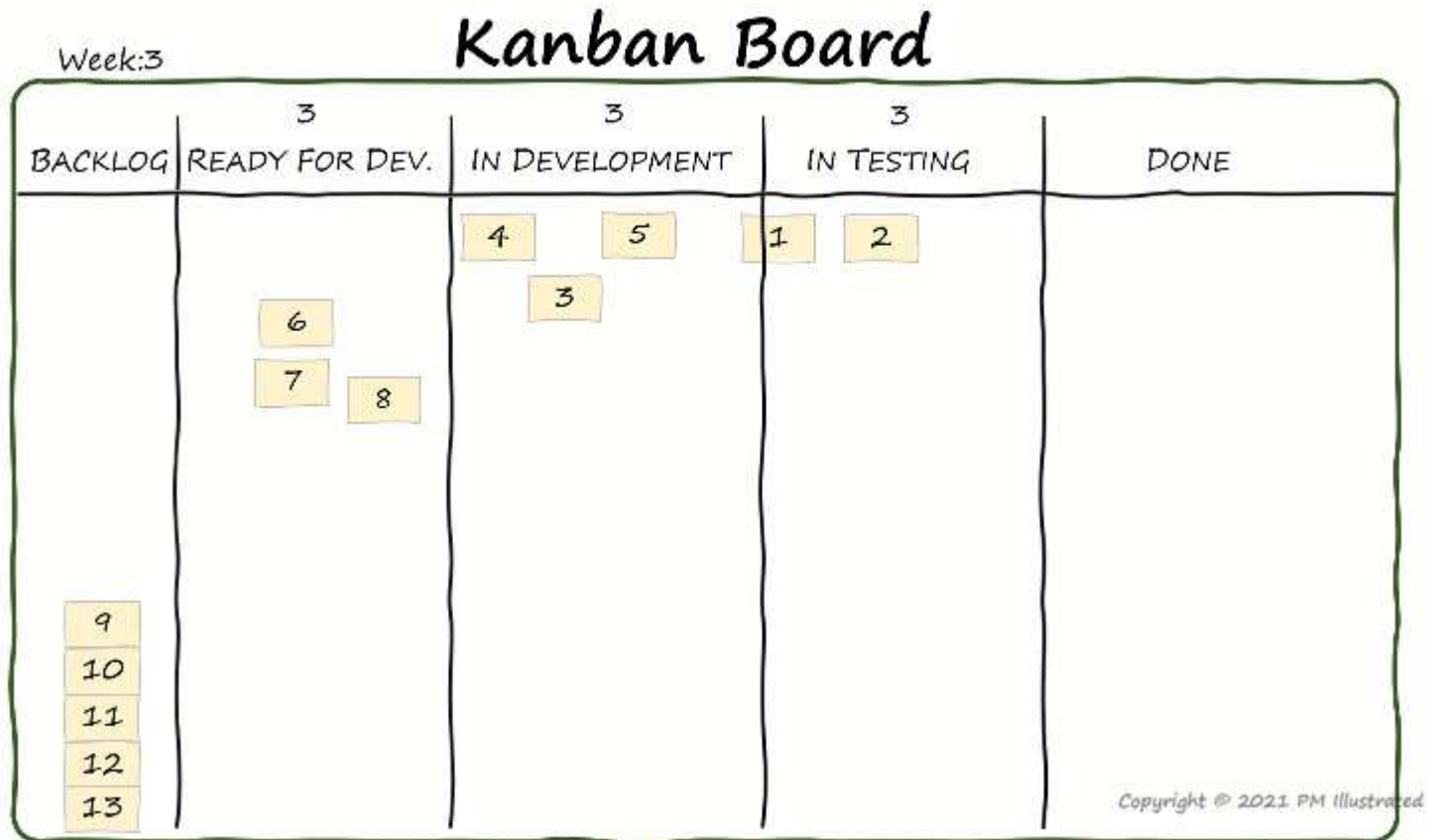
Esempio di funzionamento - 2



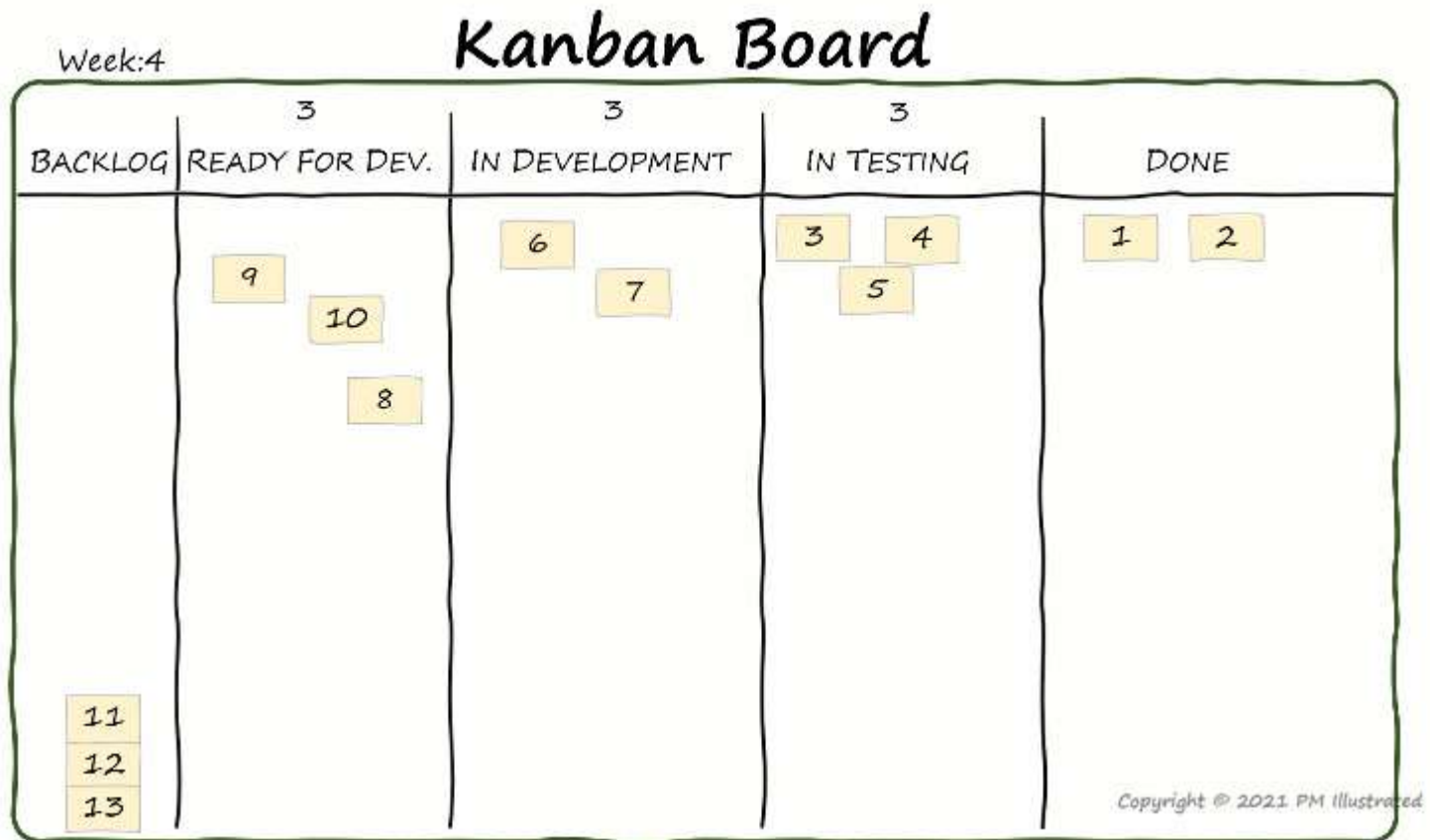
Esempio di funzionamento - 3



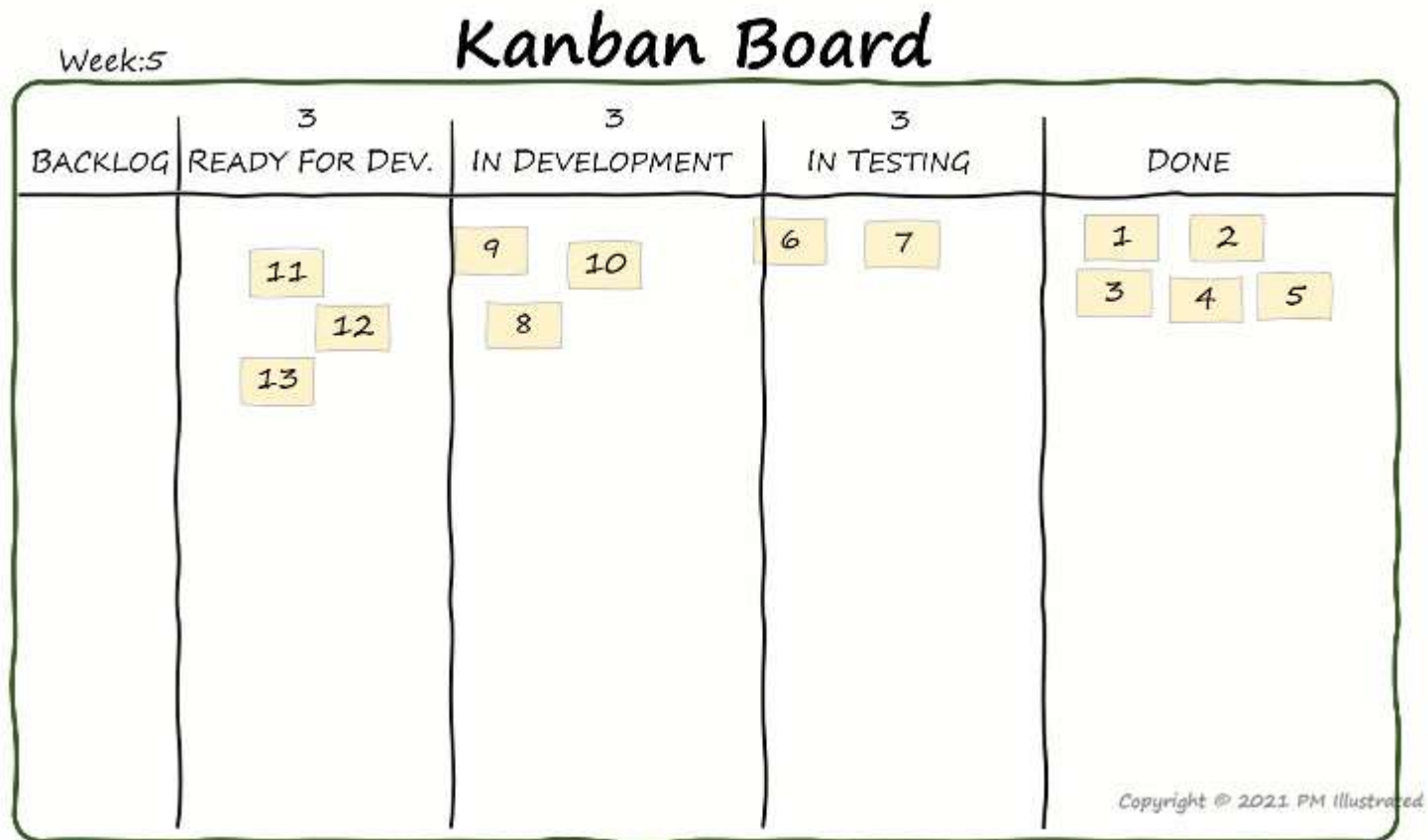
Esempio di funzionamento - 4



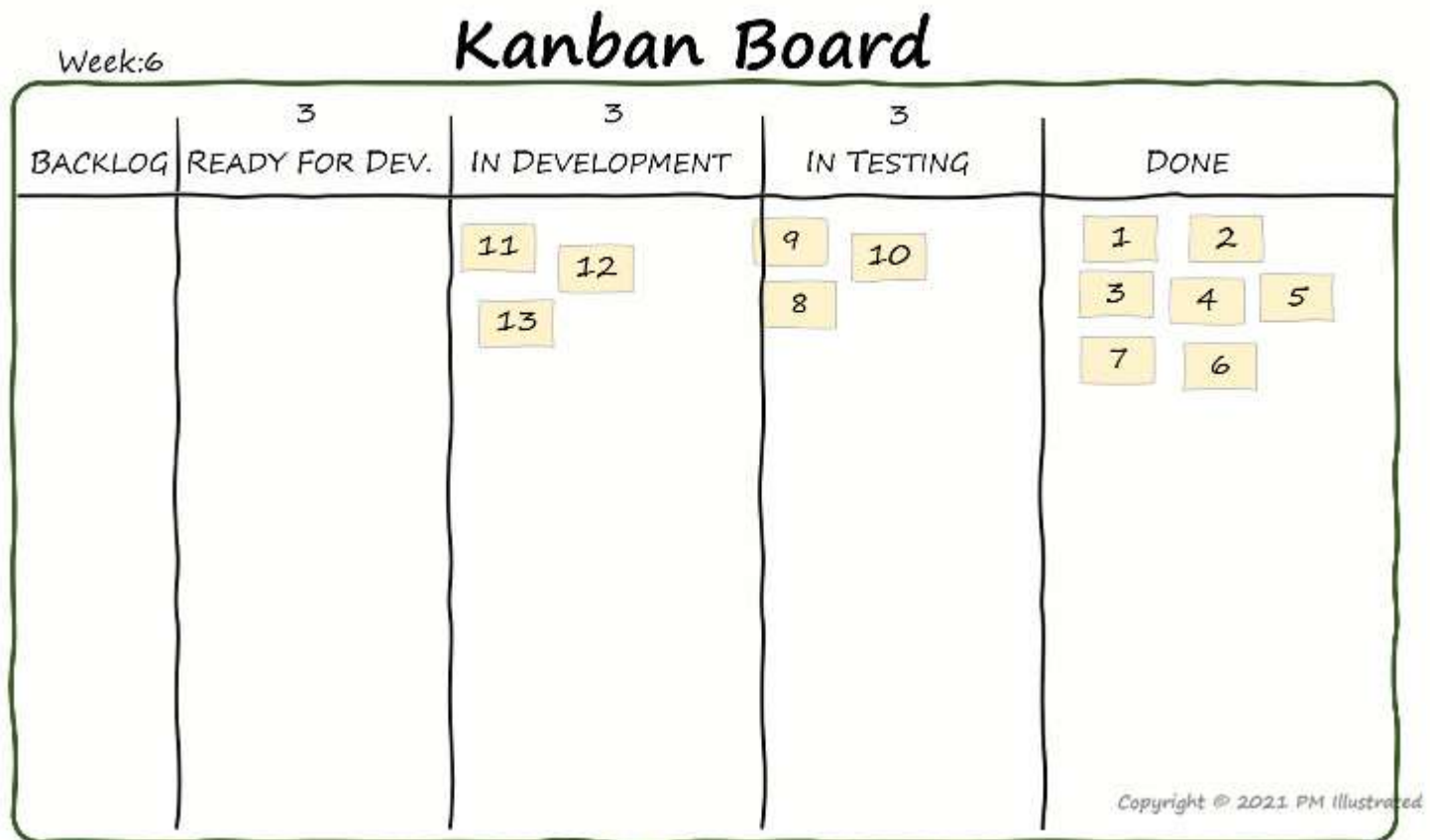
Esempio di funzionamento - 5



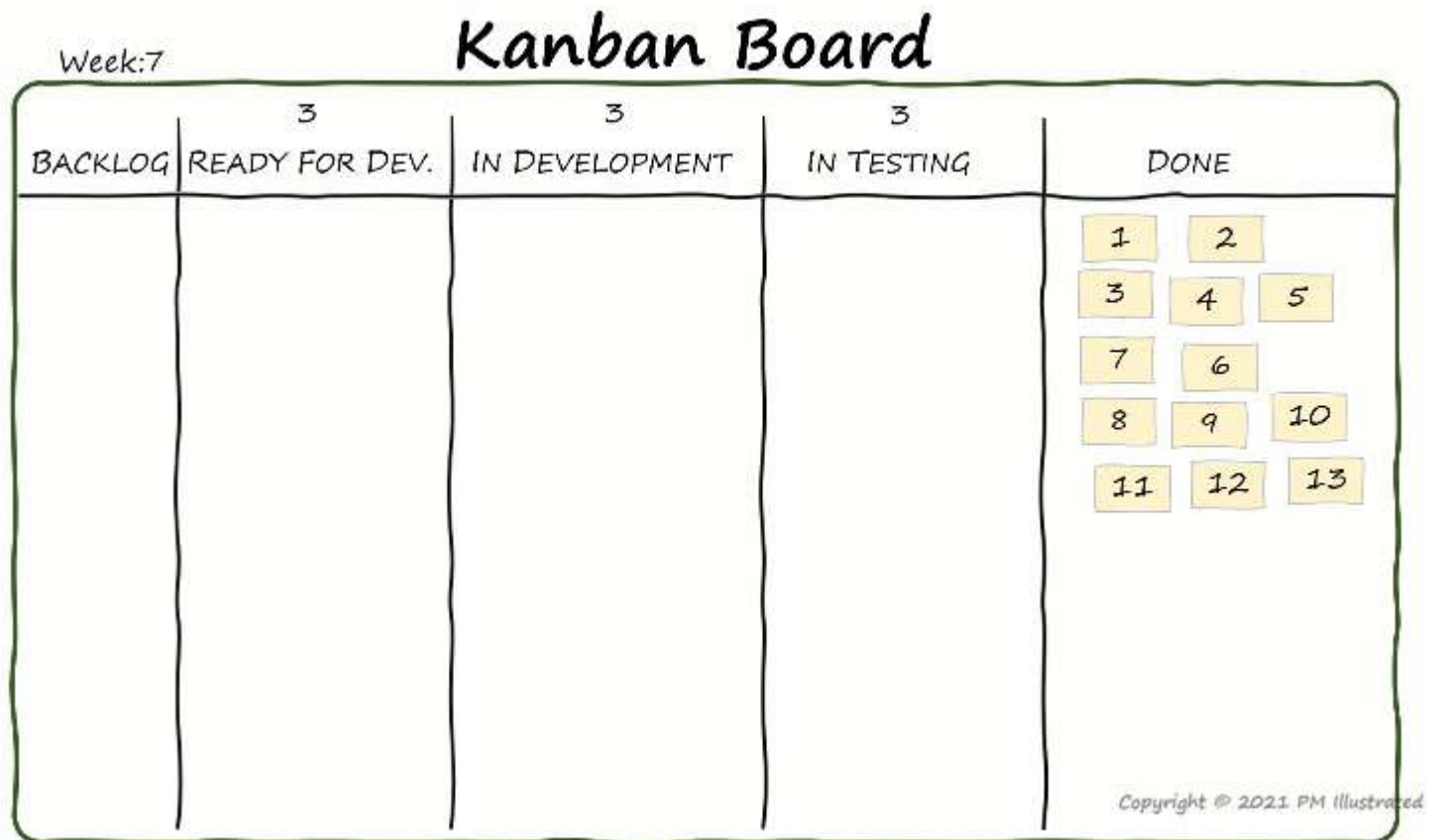
Esempio di funzionamento - 6



Esempio di funzionamento - 7



Esempio di funzionamento - 8



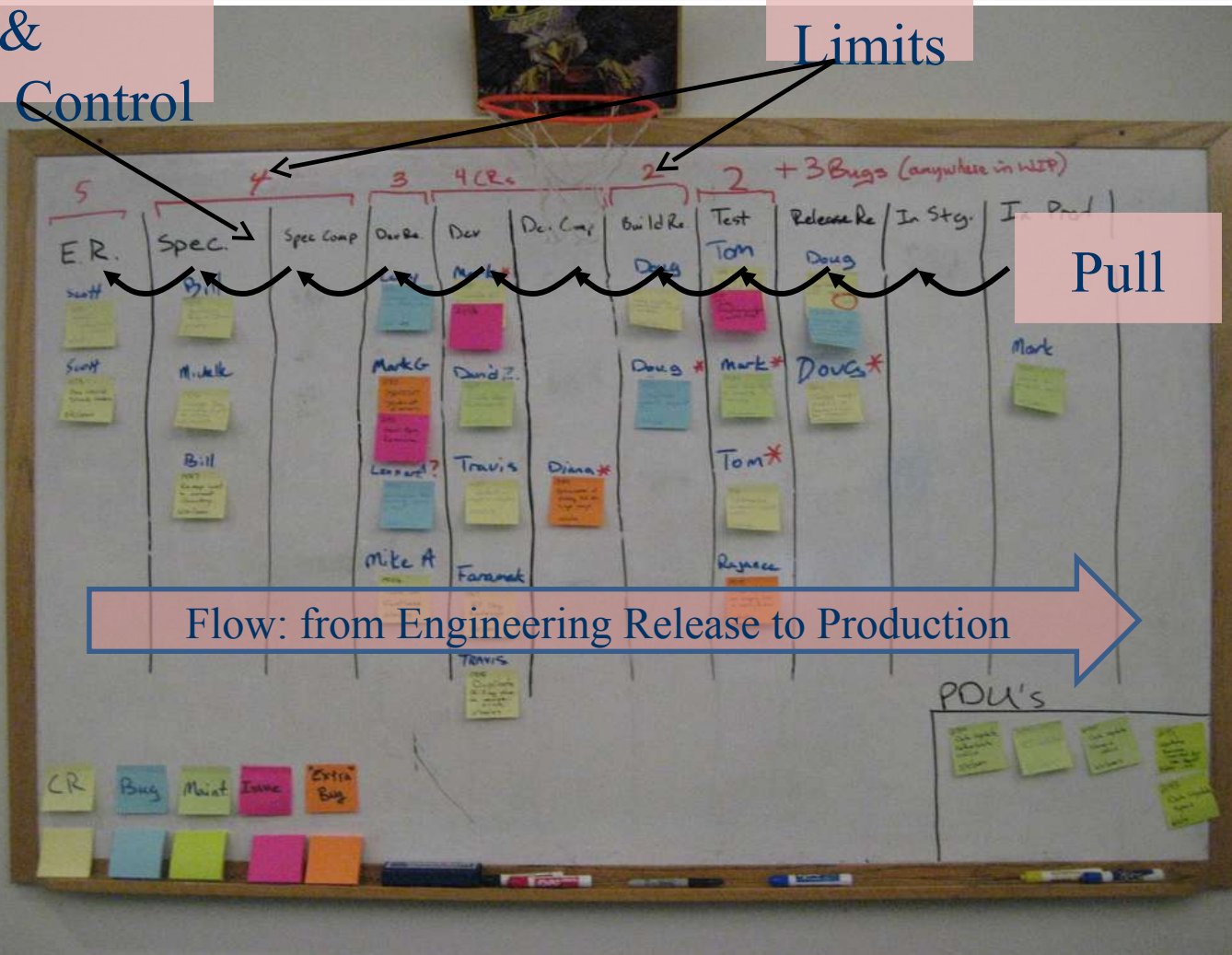
Un tabellone Kanban reale

Model
&
Visual Control

WIP
Limits

Pull

Flow: from Engineering Release to Production



Tabellone a due livelli (Two-tier)



Tabellone a due livelli (Two-tier)

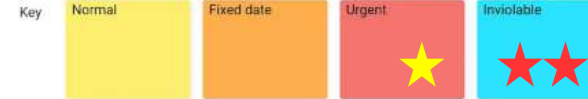
L'osservazione del tabellone evidenzia subito quali attività siano i colli di bottiglia

Their Stories

Un altro tabellone (virtuale)

Kanban Board

Flight reservation platform



Backlog 1/2

ProjectId_54
Activities search bar
Research

ProjectId_55
Customer support through Messenger
Account

ProjectId_56
Android App
Account

ProjectId_57
iOS App
Account

ProjectId_58
Airline review
Research

ProjectId_59
Maximum stop duration filter
Research

ProjectId_60
10 minutes remaining popup
Account

ProjectId_61
Adding travelers once flight found
Account

ProjectId_62
Flight+Hotel search bar
Research

ProjectId_64
Sharing deal
Offer

Backlog 2/2

ProjectId_50
Yield management
Price&Payment

ProjectId_51
SMS confirmation 24h before departure
Account

ProjectId_52
Flight research time optimization
Research

ProjectId_53
Arrival time feature
Research

	12 Selected	7 To specify		6 In development		2 To test	3 To deploy in production	
		In progress	Done	In progress	Done		In progress	Done
Flights	ProjectId_49 Destination to anywhere Offer ProjectId_39 Facebook SSO Account ProjectId_48 Airline filter Research	ProjectId_37 Show public holidays in local countries Research ProjectId_38 Adding several travelers to a unique account Account ProjectId_40 Historical flight activity (5 years back) Account	ProjectId_36 Price alert feature Price&Payment	ProjectId_38 Multi-city flights Urgent ProjectId_29 Show number of tickets left Offer	ProjectId_25 Seat selection Offer ProjectId_26 Meal selection Offer	ProjectId_23 Adapt cookies to GDPR Urgent Account ProjectId_24 Paypal integration Price&Payment	ProjectId_32 Google SSO Inviolable Account	
Other services	ProjectId_43 Car rental special offers Offer	ProjectId_42 Car rental Offer ProjectId_41 Shuttle and transfer information Research			ProjectId_27 Proposing accommodation at arrival place Offer			ProjectId_21 Hotel search bar Research
Bugs	ProjectId_45 Connection airports display Research	ProjectId_44 Currency conversion Price&Payment ProjectId_46 Autocompletion with special characters Research	ProjectId_33 Check-in baggage information Account	ProjectId_30 Multi-page flight display Offer ProjectId_32 Number of tickets left error Offer	ProjectId_31 Integrity of Air France data Urgent			
Fridge		ProjectId_47 Privacy policy update Account		ProjectId_34 Vaccine passport upload Account ProjectId_35 Last minute offers Offer				

Tabellone Kanban virtuale (Trello)

The screenshot displays a Trello Kanban board titled "Product Development". The board is organized into four columns: "In progress", "Testing", "Tested", and "Code Review". Each column contains several task cards, each with a title, a progress bar, a comment count, and an assignee's profile picture.

- In progress:**
 - EditableFieldView (Assignee: [Profile])
 - Fix markAsViewed logic (Assignee: [Profile])
 - Renderable Action (Assignee: [Profile])
 - Renderable Detail View (Assignee: [Profile])
 - New iOS Design (Progress: 50%, Comment: 2, Assignee: [Profile])
 - J/K Keyboard Shortcuts Disabled (Progress: 20%, Comment: 4, Assignee: [Profile])
 - iOS App Login Screen Wireframes (Progress: 10%, Assignee: [Profile])
- Testing:**
 - Attachment preview icon (Comment: 3, Assignee: [Profile])
 - Don't Display Private Documents To Team (Comment: 1, Assignee: [Profile])
 - Clicking icon should open pop-over (Assignee: [Profile])
- Tested:**
 - Let the server choose the default name when creating a file from a URL (Progress: 100%, Comment: 3, Assignee: [Profile])
 - Attach URLs from comment (Comment: 2, Assignee: [Profile])
 - Plugins (Comment: 1, Assignee: [Profile])
 - Update CSS (Comment: 8, Assignee: [Profile])
- Code Review:**
 - Plugin enable/disable actions (Comment: 3, Assignee: [Profile])
 - New Diagnostics (Comment: 1, Assignee: [Profile])
 - Pre-load attachments (Comment: 1, Assignee: [Profile])
 - Add post-message-io (Comment: 3, Assignee: [Profile])

The interface includes a top navigation bar with "Boards", a search bar, the Trello logo, and user avatars. The board header shows "Product Development", "Travidux", and "Team Visible". The bottom of the board features a "Calendar" and "Show Menu" link.

Gestione del team

- ◆ Non c'è un approccio specifico per la gestione del team e delle competenze individuali, ma si usano approcci mutuati da altre metodologie, agili e no
- ◆ Poiché la lavorazione delle feature è fatta in attività specifiche, si può pensare sia a specialisti che svolgono esclusivamente un tipo di attività (ad es. il testing o la progettazione), oppure a sviluppatori generalisti capaci di scegliere l'attività in quel momento più critica.



Gestione del team

- ◆ La dimensione di un team che segue un approccio Kanban può essere anche sino a 50-70 persone, ben maggiore di quella dei tipici team Scrum o XP
 - In caso di team grandi, i ruoli saranno tipicamente più specializzati che nel caso di team di 5-10 persone
- ◆ La scelta di quali siano le specifiche feature in una certa fase su cui lavorare è lasciata agli sviluppatori, anche se è consigliabile che uno sviluppatore, una volta iniziata l'attività su una o più feature, le porti a termine prima di iniziarne altre

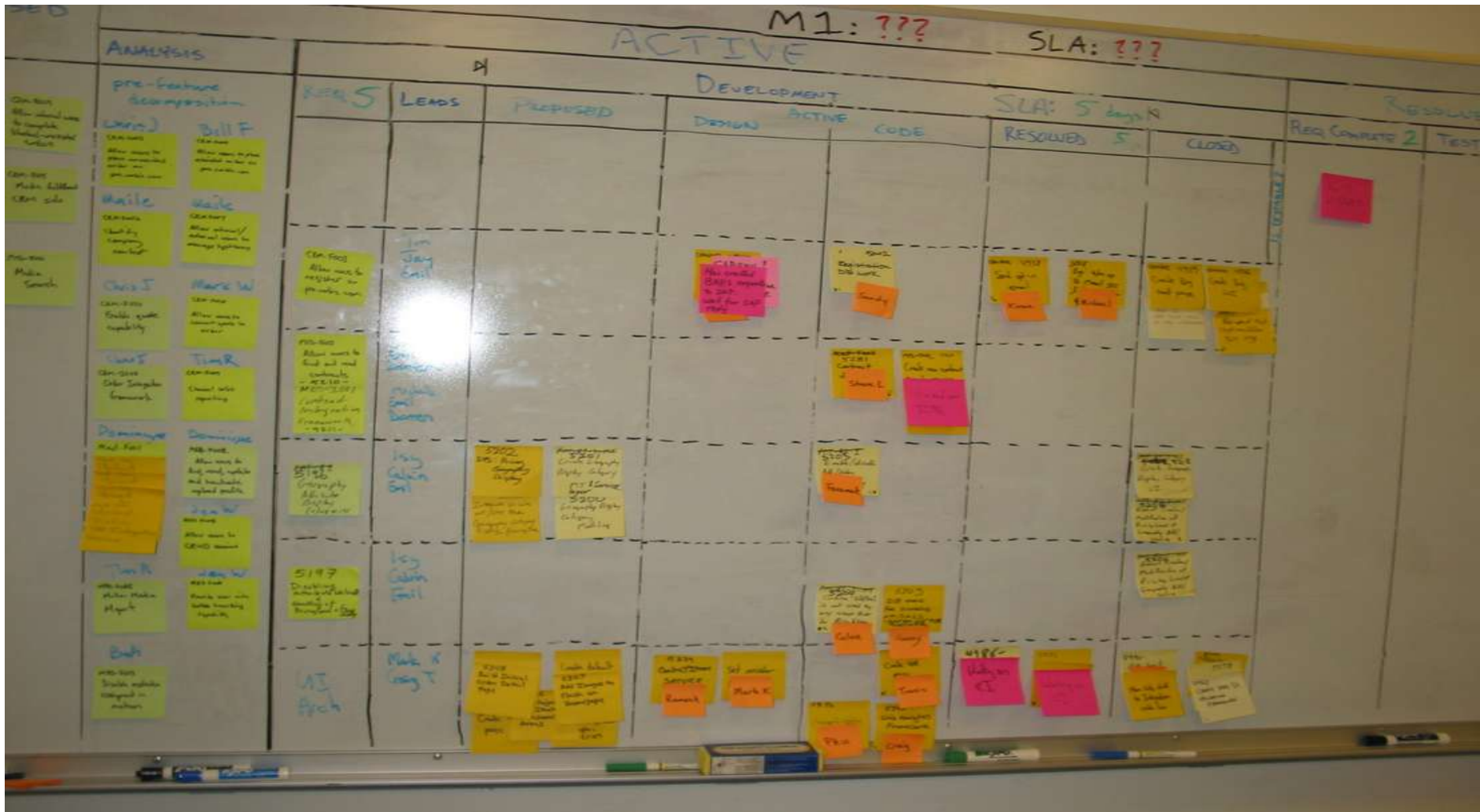


Gestione di più progetti

- ♦ L'approccio Kanban è compatibile con lo sviluppo di più progetti in parallelo, perché le singole feature possono appartenere anche a progetti diversi.
 - In tal caso, si usano schede di colore diverso, o si tracciano “corsie” orizzontali sul tabellone, riferite ai diversi progetti.
- ♦ Si possono anche facilmente gestire:
 - la scomposizione delle feature in “task” più semplici
 - sotto-attività specifiche per ogni attività – come ad es. la progettazione dell'interfaccia, del database, l'implementazione della “business logic” e delle politiche di sicurezza
 - ...ed altro ancora



Gestione di più di un progetto: swimlanes (corsie)

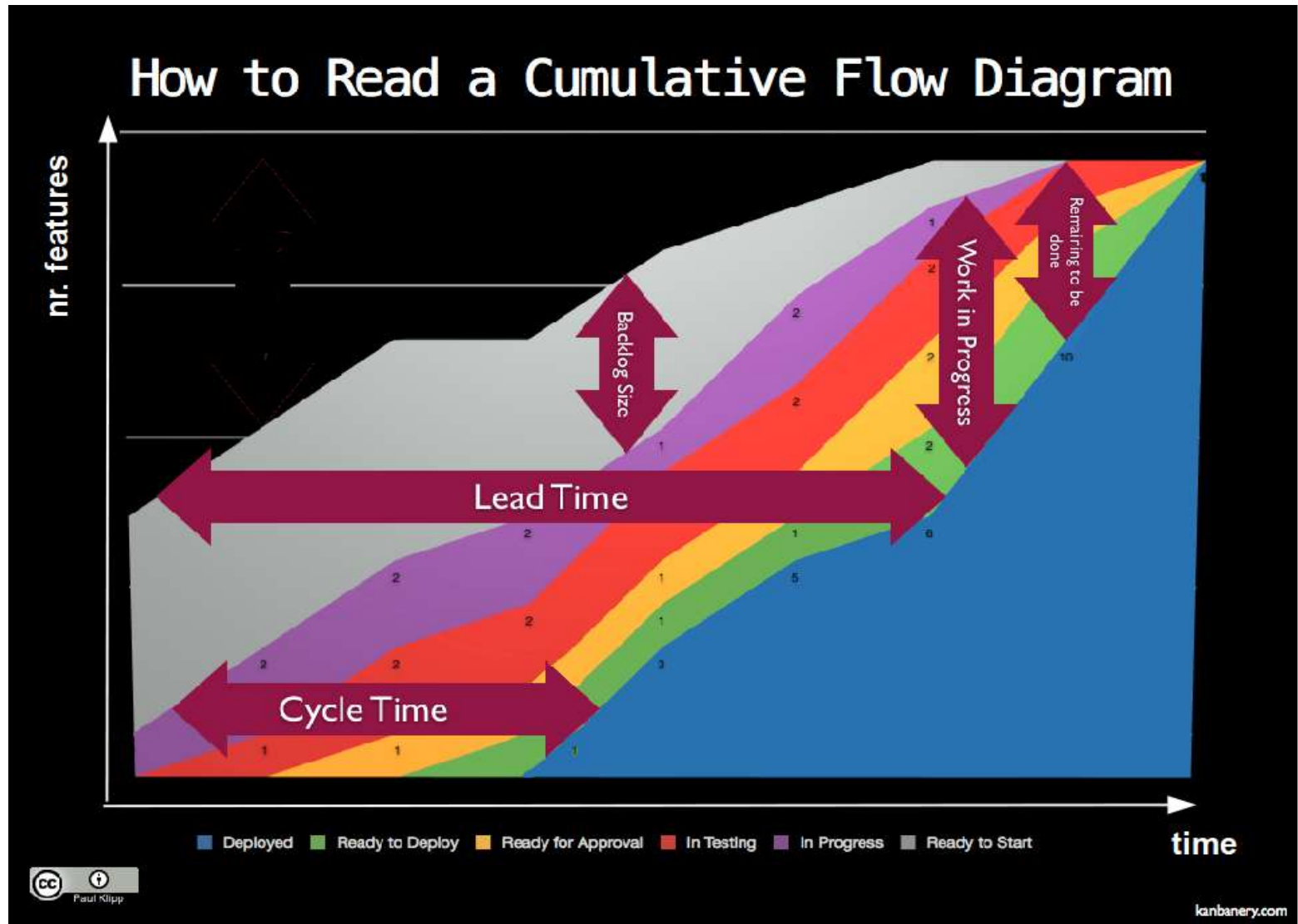


Cumulative Flow Diagram (CFD)

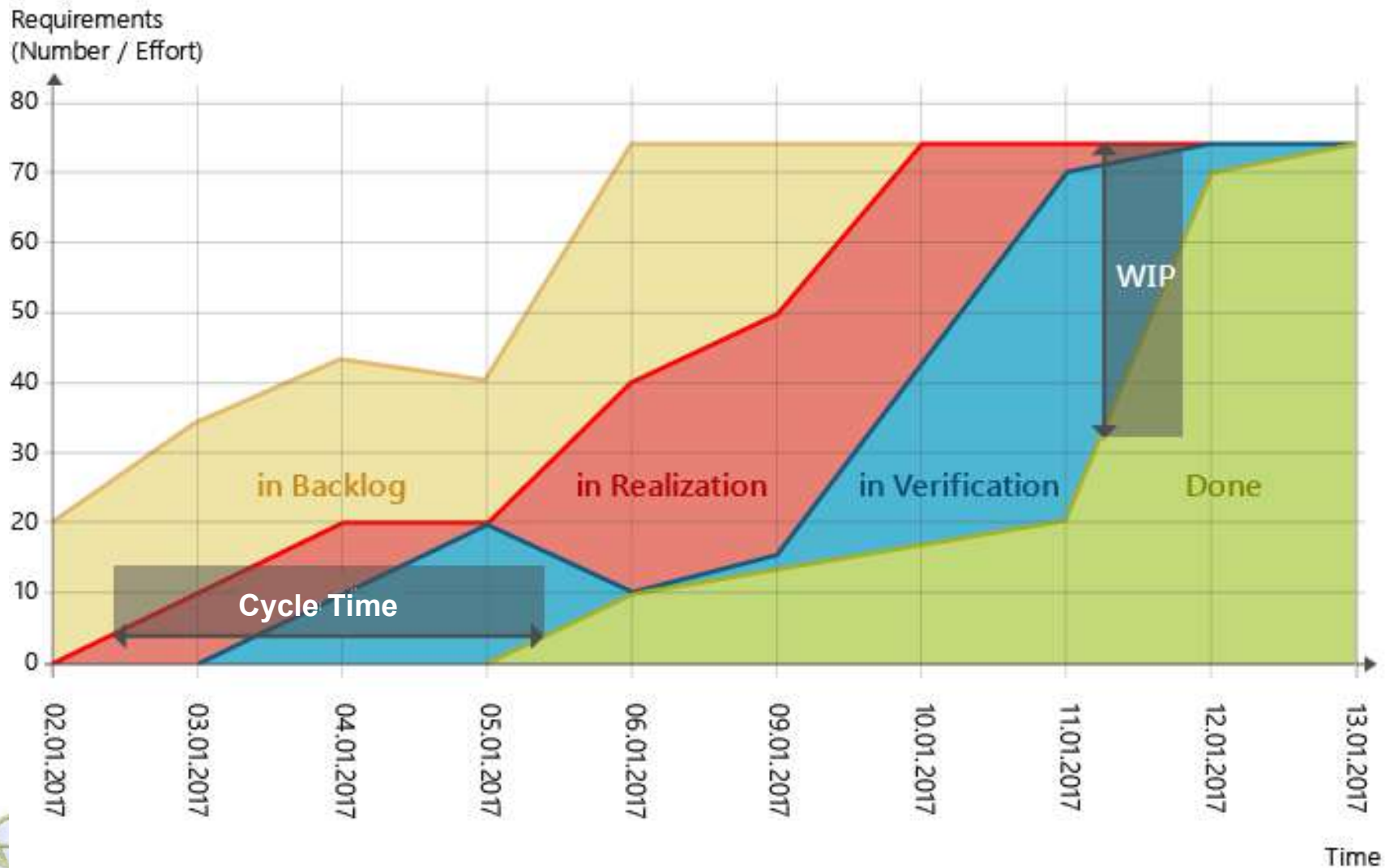
- ♦ Il CFD mostra in ascisse il tempo, e in ordinate il numero complessivo di tutte le feature **completate** in ogni attività, con una linea (crescente) per ogni attività
- ♦ Le attività che avvengono prima hanno linee più in alto
- ♦ Le zone tra le linee sono colorate per evidenziarle
- ♦ La linea più in alto mostra il nr. delle feature aggiunte al backlog
- ♦ Lo spessore verticale di ogni zona rappresenta il numero di feature in lavorazione nell'attività, o il numero di feature completate nella zona più in basso
- ♦ Lo spessore orizzontale rappresenta il tempo di transito in un'attività o in un insieme di attività



Cumulative Flow Diagram (CFD)

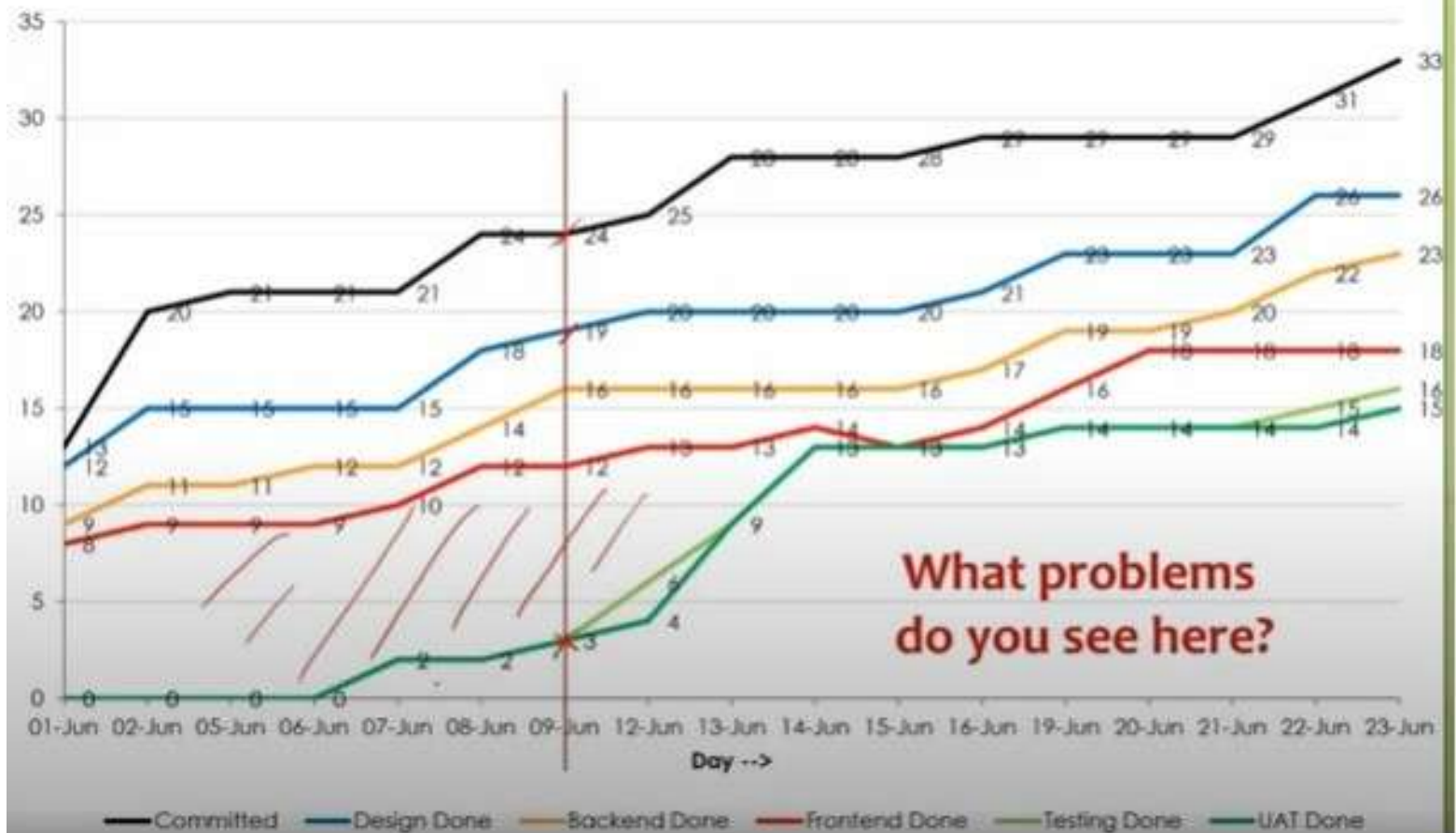


Un altro CFD



Usare il CFD per evidenziare problemi

Sample CFD



Usare il CFD per evidenziare problemi

- ♦ Se gli spazi tra due linee sono molto distanti verticalmente significa che c'è troppo work in progress tra le due attività (zona tratteggiata tra le linee rossa e verde):
 - c'è chiaramente un collo di bottiglia, che impedisce al flusso di progredire
- ♦ Se due linee sono troppo vicine o si toccano, significa che c'è poco o nessun lavoro in corso nell'attività in alto, come le linee verde chiaro e verde scuro (User Acceptance Testing)
- ♦ Se una linea è quasi piatta significa che NESSUN lavoro è stato completato in quei giorni – ciò implica che la produttività tende quasi a zero.
- ♦ Nel CFD la distanza tra due linee vicine dovrebbe essere ottimale: né troppo grande né troppo piccola, e le linee dovrebbero essere sempre crescenti (tranne il backlog)!

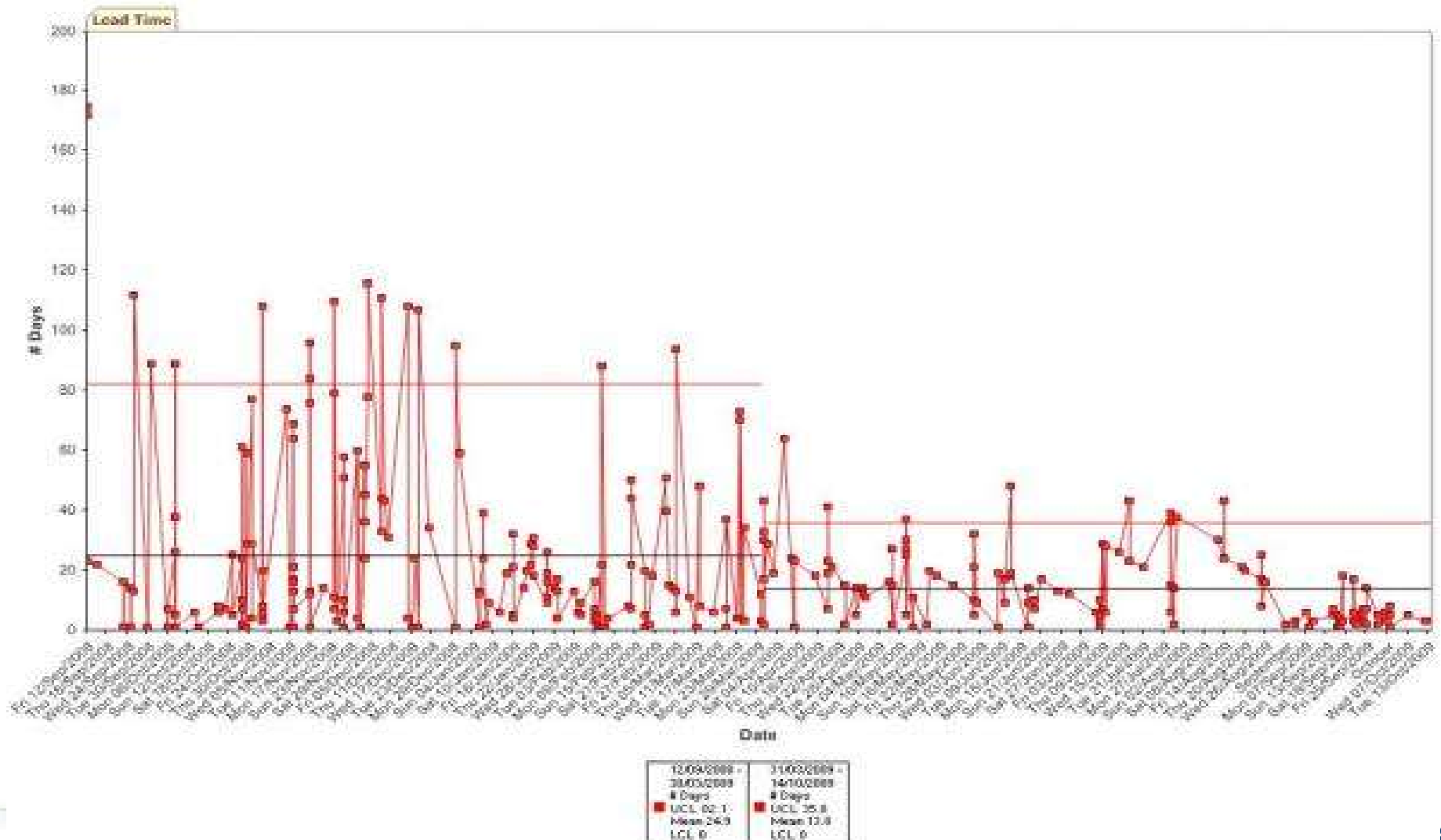


Usare il CFD (<https://www.youtube.com/watch?v=eo2uv8avEsU>)

Cumulative Flow Diagram
The basics in 2 minutes



Lead times analysis



Kanban Method

- ◆ Un processo che segue l'approccio Kanban è composto dalle seguenti attività:
 1. Attività preliminare di raccolta dei requisiti per determinare e valutare un insieme di feature o US da cui iniziare lo sviluppo del sistema
 2. Attività preliminare per determinare l'architettura base del sistema
 3. Sviluppo per feature/US, con approccio pull usando il tabellone Kanban. Tale sviluppo è asincrono e non iterativo
 4. Stand-up meeting giornaliero di fronte al tabellone Kanban, per evidenziare gli eventuali problemi e “blocchi”



Kanban Method

5. Ogni settimana, si scelgono nuove feature/US dal backlog e le si mettono in produzione, rispettando il WIP della colonna “Ready for Dev.” o equivalente.
6. Riunioni di pianificazione, con raccolta di feedback da parte dell'utente e revisione/aggiunta delle feature del sistema. **Non hanno una periodicità fissa, ma sono convocate quando necessario.**
7. Riunioni retrospettive di verifica del lavoro in corso, determinazione degli ostacoli, miglioramenti al processo (come in Scrum).
8. Gestione dei rilasci a tempi prefissati, con eventuale negoziazione delle feature previste per ogni rilascio



Kanban vs. Scrum

- ◆ Entrambi i metodi sono **agili** e **incrementali**:
 - Implementano Features o US di tipo INVEST
- ◆ Scrum è basato su iterazioni (Sprint)
- ◆ Kanban è a flusso continuo
- ◆ Scrum prescrive riunioni prima e dopo lo Sprint
- ◆ Kanban non ha riunioni di pianificazione prefissate
- ◆ Entrambi hanno la riunione in piedi giornaliera
- ◆ Entrambi usano un tabellone per mostrare il progresso
- ◆ Scrum ha ruoli specifici (Scrum Master, Product Owner)
- ◆ Kanban non ha ruoli specifici
- ◆ Entrambi non prescrivono pratiche specifiche di progetto e codifica, ma in genere sono usate le pratiche agili



Kanban vs. Scrum

Aspetto	Scrum	Kanban
Iterazioni	Sprint fissi (1-4 settimane)	Flusso continuo
Ruoli	PO, SM, Team	Nessun ruolo prescritto
Artifact	Product/Sprint Backlog	Kanban Board
Cerimonie	Sprint Planning, Daily Scrum, Sprint Review, Retrospettiva	Riunione giornaliera, Replenishment, Retrospettiva (flessibili)
Stime	Story Points, Project Velocity	Cycle Time, Throughput
Cambiamenti	Mai durante lo sprint	In qualsiasi momento (se c'è capacità)



Scrumban

- ◆ Scrumban usa la struttura di Scrum con le pratiche di flusso di Kanban.
- ◆ Ad esempio:
 - Sprint di 2 settimane (Scrum)
 - Board con limiti WIP per colonna (Kanban)
 - Metriche: Project Velocity + Cycle Time
- ◆ Vantaggi:
 - Più strutturato rispetto a Kanban puro
 - Migliore per iniziare a usare Kanban
 - Molto adatto a sviluppi di un solo progetto
 - Adatto per progetti con scadenze temporali precise



Conclusioni sul Kanban

- ◆ È un approccio di gestione del processo neutrale rispetto alle specifiche pratiche di sviluppo
- ◆ È un approccio **WIP-limited**, che assicura un **flusso costante** di features completate
- ◆ **Non è basato su iterazioni**, permette di risparmiare le **perdite di tempo** date dalle pianificazioni delle iterazioni e dalle riunioni al loro termine
- ◆ È molto adatto alla gestione della manutenzione e per i team che lavorano su 2 o più progetti
- ◆ È adatto per team multi-sito (attraverso l'utilizzo di un tabellone Kanban virtuale)

